

Documentation Completeness Agent - Requirements Document

Project Overview

Purpose

Build an AI-powered agent that automatically analyzes code repositories to identify missing, incomplete, or outdated documentation. The system will generate documentation drafts, maintain documentation currency as code changes, and provide actionable insights to improve documentation quality across Softura's development teams.

Business Context

- **Problem:** Poor documentation contributes to Softura's 40% rework rate and knowledge silos
- **Impact:** Reduces onboarding time, improves knowledge transfer, supports COO succession planning
- **Strategic Alignment:** Part of Softura's AI adoption and operational excellence transformation

Target Users

1. Development team leads and managers
 2. Individual developers contributing code
 3. QA teams needing test documentation
 4. Project managers tracking documentation health
 5. New hires requiring onboarding materials
-

Functional Requirements

FR-1: Repository Analysis

Priority: P0 (Must Have)

The system shall analyze code repositories from GitHub and Azure DevOps to assess documentation completeness.

Capabilities:

- Connect to GitHub repositories via GitHub API
- Connect to Azure DevOps repositories via ADO API
- Support multiple programming languages (.NET/C#, JavaScript/TypeScript, Python, Java)
- Scan entire repositories or specific branches

- Process incremental changes (delta analysis)
- Handle repositories up to 100,000 files

Acceptance Criteria:

- ✓ Successfully authenticates with GitHub and ADO using OAuth/PAT tokens
- ✓ Retrieves complete file tree for a given repository
- ✓ Filters files by type (.cs, .js, .ts, .py, .java, .md, etc.)
- ✓ Processes a 10,000-file repository in under 5 minutes
- ✓ Handles rate limiting gracefully with retry logic
- ✓ Logs all API calls and errors for troubleshooting

FR-2: Code Documentation Analysis

Priority: P0 (Must Have)

The system shall analyze code files to identify documentation gaps and quality issues.

Analysis Types:

1. Class/Interface Documentation

- Presence of XML documentation comments (C#)
- JSDoc comments (JavaScript/TypeScript)
- Docstrings (Python)
- Completeness of parameter descriptions
- Return value documentation
- Exception documentation

2. Method/Function Documentation

- Summary descriptions
- Parameter documentation
- Return type documentation
- Example usage
- Side effects and warnings

3. File-level Documentation

- Copyright headers
- Module/namespace descriptions
- Purpose and responsibility statements

4. **Public API Documentation**

- All public methods documented
- All public properties documented
- API contracts clearly defined

Acceptance Criteria:

- ✓ Identifies classes without XML docs/JSDoc/docstrings
- ✓ Flags methods with missing parameter documentation
- ✓ Detects undocumented public APIs
- ✓ Calculates documentation coverage percentage per file
- ✓ Identifies stale documentation (code changed, docs didn't)
- ✓ Generates severity levels (Critical, High, Medium, Low)

FR-3: README and Markdown Analysis

Priority: P0 (Must Have)

The system shall evaluate repository README files and markdown documentation.

Analysis Points:

- README.md presence and completeness
- Essential sections (Overview, Installation, Usage, Configuration)
- Code examples and snippets
- Architecture diagrams or references
- API documentation links
- Contributing guidelines
- License information
- Contact/support information

Acceptance Criteria:

- ✓ Identifies missing README.md files
- ✓ Checks for minimum required sections
- ✓ Validates links are not broken (HTTP checks)
- ✓ Identifies outdated version references
- ✓ Suggests missing sections based on project type
- ✓ Scores README quality (0-100 scale)

FR-4: AI-Powered Documentation Generation

Priority: P0 (Must Have)

The system shall use AI to generate documentation drafts for undocumented code.

Capabilities:

- Generate XML documentation comments for C# classes/methods
- Generate JSDoc comments for JavaScript/TypeScript
- Generate Python docstrings
- Create README.md templates
- Generate API documentation from code analysis
- Suggest improvements to existing documentation

Quality Requirements:

- Generated docs must be technically accurate
- Must include all parameters and return types
- Must identify and document exceptions/error conditions
- Must be in the correct format for the language
- Must maintain consistent tone and style

Acceptance Criteria:

- ✓ Generates accurate documentation for 90%+ of simple methods
- ✓ Produces correct XML/JSDoc/docstring syntax
- ✓ Includes all method parameters in generated docs
- ✓ Documents return types accurately
- ✓ Flags complex methods needing human review
- ✓ Allows developers to accept/reject/edit suggestions

FR-5: Documentation Health Dashboard

Priority: P0 (Must Have)

The system shall provide a web-based dashboard showing documentation health metrics.

Dashboard Views:

1. **Executive Summary**

- Overall documentation coverage (%)
- Trend over time (improving/declining)
- Top 10 least documented repositories
- Critical gaps requiring immediate attention

2. Repository Details

- Per-repository coverage percentage
- Breakdown by file type
- Recent changes impact on documentation
- Historical trends

3. Team/Developer View

- Documentation quality by team/individual
- Contribution to documentation improvements
- Gamification elements (badges, streaks)

4. Issue Tracking

- List of all documentation gaps
- Severity and priority
- Assignment to developers
- Resolution status

Acceptance Criteria:

- ✓ Dashboard loads in under 3 seconds
- ✓ Real-time data refresh available
- ✓ Exportable reports (PDF, Excel)
- ✓ Filterable by repository, team, language, severity
- ✓ Drill-down from summary to file-level detail
- ✓ Mobile-responsive design

FR-6: Azure DevOps Integration

Priority: P0 (Must Have)

The system shall integrate seamlessly with Azure DevOps workflows.

Integration Points:

1. Work Item Creation

- Auto-create ADO work items for documentation gaps
- Link work items to specific code files
- Assign to appropriate developers
- Set priority based on severity

2. Pull Request Integration

- Analyze documentation changes in PRs
- Comment on PRs with documentation feedback
- Block PRs if critical documentation missing (configurable)
- Show documentation coverage delta

3. Pipeline Integration

- Run as part of CI/CD pipelines
- Generate documentation reports
- Fail builds if coverage drops below threshold
- Publish results to pipeline artifacts

Acceptance Criteria:

- ✓ Creates ADO work items with correct fields populated
- ✓ Links work items to source control paths
- ✓ Posts comments on PRs within 2 minutes of PR creation
- ✓ Pipeline task completes in under 10 minutes
- ✓ Reports published to ADO artifacts
- ✓ Configurable quality gates and thresholds

FR-7: Notification and Alerting

Priority: P1 (Should Have)

The system shall notify stakeholders of documentation issues.

Notification Types:

- Email alerts for critical gaps
- Slack/Teams integration for team notifications
- Weekly summary reports to managers
- Real-time alerts for coverage drops

Acceptance Criteria:

- ✓ Sends email notifications within 5 minutes of detection
- ✓ Configurable notification preferences per user
- ✓ Digest mode available (daily/weekly summaries)
- ✓ Unsubscribe mechanism available
- ✓ Notifications include actionable links

FR-8: Documentation Templates and Standards

Priority: P1 (Should Have)

The system shall support organizational documentation standards.

Capabilities:

- Define custom documentation templates
- Enforce organizational style guidelines
- Support multiple documentation standards (Microsoft, Google, custom)
- Template library for common scenarios

Acceptance Criteria:

- ✓ Admin can upload custom templates
- ✓ Templates applied based on file type/project type
- ✓ Style guide violations flagged
- ✓ Developers can preview template output

FR-9: Historical Tracking and Trends

Priority: P1 (Should Have)

The system shall track documentation health over time.

Metrics to Track:

- Documentation coverage percentage over time
- Number of gaps created vs. resolved
- Time to resolution for documentation issues
- Developer contributions to documentation
- Repository health trends

Acceptance Criteria:

- ✓ Stores historical data for minimum 2 years
- ✓ Generates trend charts and visualizations
- ✓ Exports historical data for analysis
- ✓ Compares team/repository performance

FR-10: Search and Discovery

Priority: P2 (Nice to Have)

The system shall enable searching across all documentation.

Search Capabilities:

- Full-text search across generated and existing docs
- Search by code element (class, method, etc.)
- Search by author or timestamp
- Semantic search using AI embeddings

Acceptance Criteria:

- ✓ Returns results in under 2 seconds
 - ✓ Ranks results by relevance
 - ✓ Highlights search terms in results
 - ✓ Supports advanced query syntax
-

Technical Specifications

Technology Stack

Backend

- **Framework:** .NET 8 (ASP.NET Core Web API)
- **Language:** C# 12
- **AI/ML Libraries:**
 - Azure OpenAI SDK (Azure.AI.OpenAI)
 - Semantic Kernel for orchestration
- **API Clients:**
 - Octokit.NET for GitHub API

- Microsoft.TeamFoundation.SourceControl.WebApi for ADO
- Microsoft.VisualStudio.Services.WebApi for ADO work items
- **Database Access:**
 - Entity Framework Core 8
 - Npgsql (PostgreSQL driver)
- **Additional Libraries:**
 - Roslyn (Microsoft.CodeAnalysis) for C# code parsing
 - Newtonsoft.Json for JSON handling
 - Serilog for logging
 - Polly for resilience and retry policies

Frontend

- **Framework:** React 18.2+
- **Language:** TypeScript 5.0+
- **UI Framework:** Material-UI (MUI) v5
- **State Management:** Redux Toolkit or Zustand
- **Data Visualization:** Recharts
- **HTTP Client:** Axios
- **Routing:** React Router v6
- **Form Handling:** React Hook Form
- **Build Tool:** Vite

Database

- **RDBMS:** PostgreSQL 15+
- **Extensions:**
 - pgvector (for semantic search if implemented)
 - pg_trgm (for fuzzy text search)
- **Hosting:** Azure Database for PostgreSQL

AI Services

- **Primary:** Azure OpenAI Service
 - Model: GPT-4 or GPT-4 Turbo
 - Deployment: Pay-as-you-go

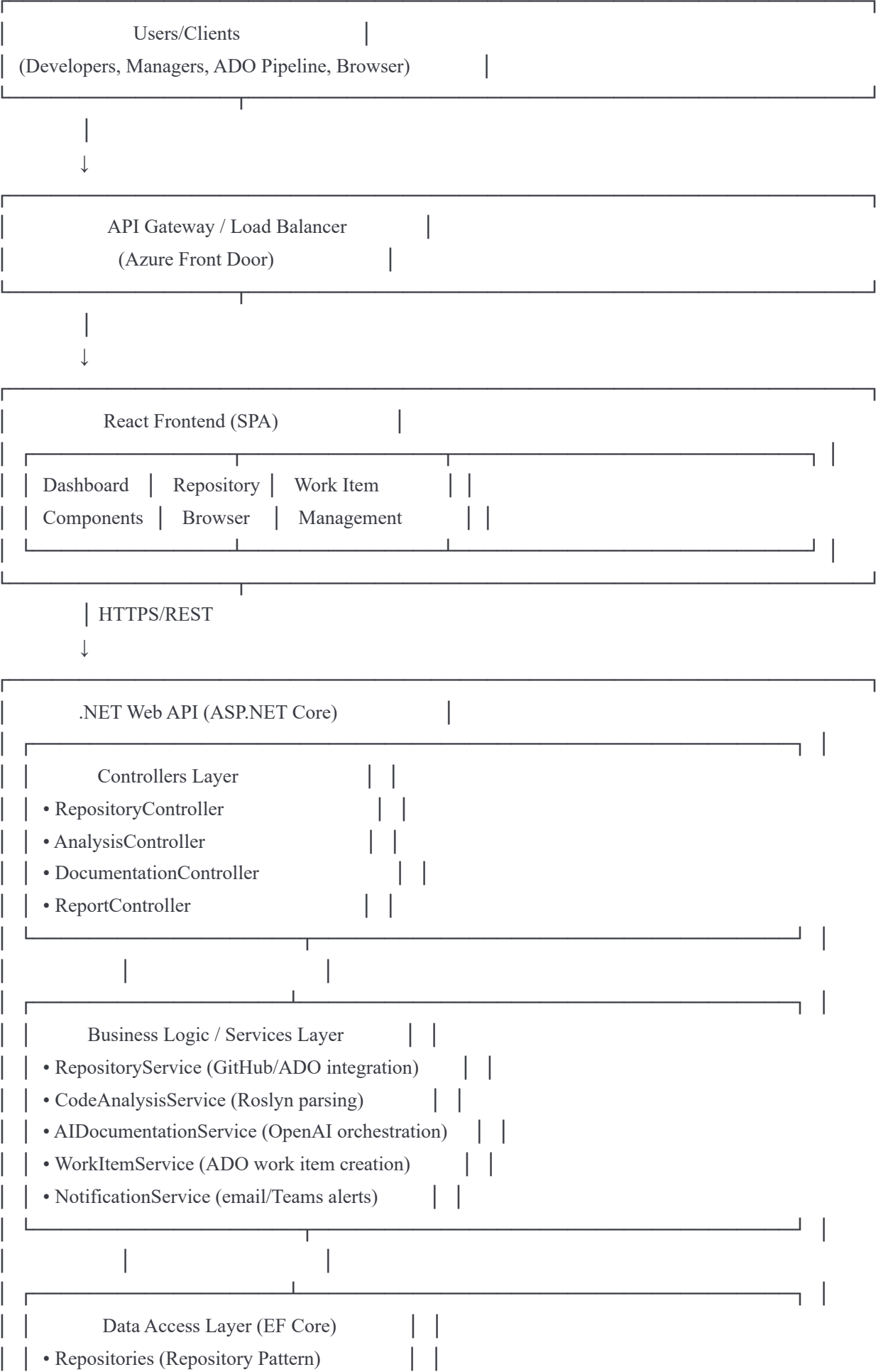
- **Embeddings** (if needed): text-embedding-ada-002

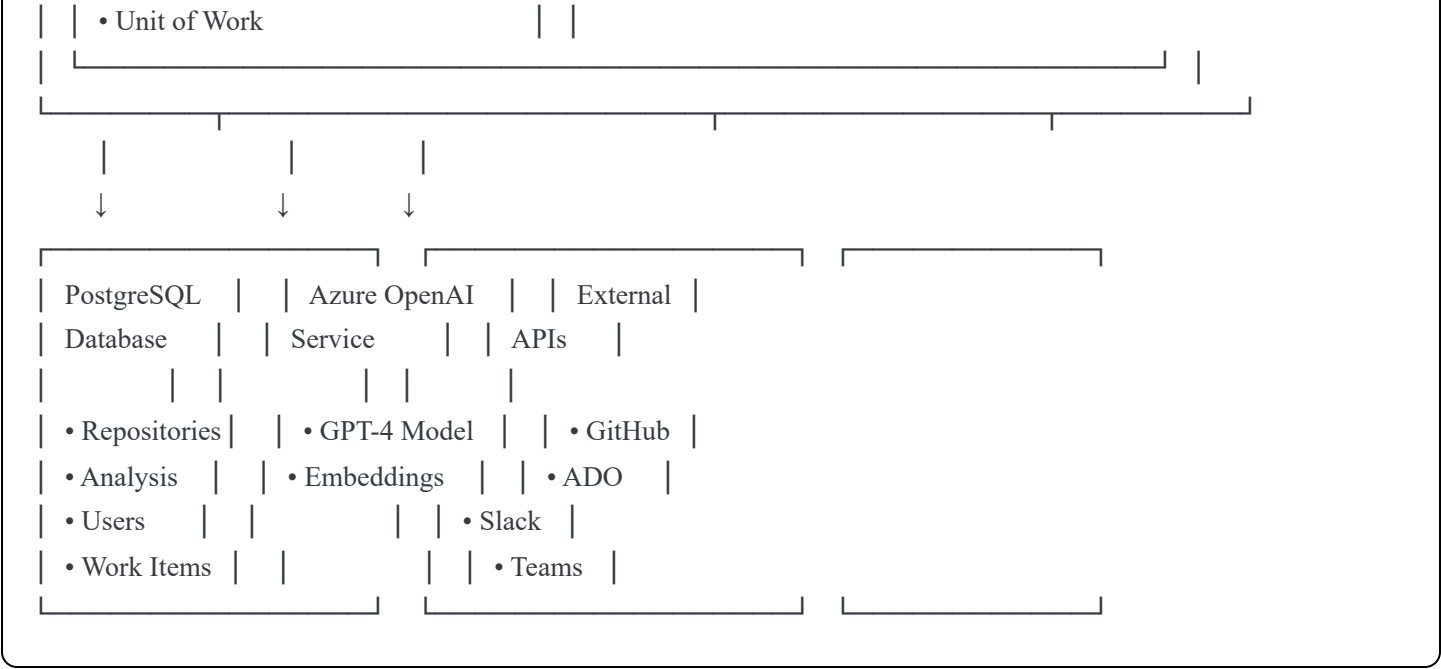
DevOps & Deployment

- **Containerization:** Docker
- **Orchestration:** Docker Compose (development), Azure Container Apps (production)
- **CI/CD:** Azure DevOps Pipelines
- **Infrastructure:** Azure App Service or Azure Container Apps
- **Monitoring:** Application Insights
- **Secrets Management:** Azure Key Vault

System Architecture

High-Level Architecture





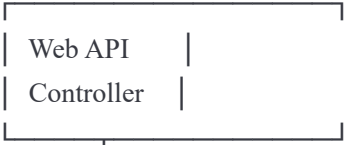
Component Diagram



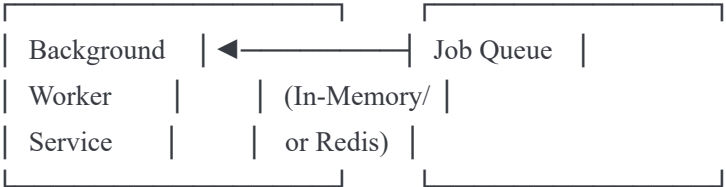
Data Flow Diagram



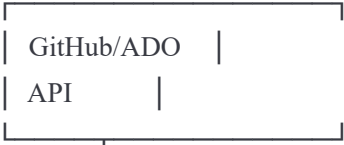
1. Request Repository Analysis



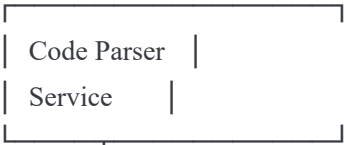
2. Queue Analysis Job



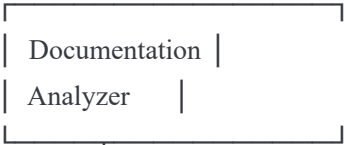
3. Fetch Repository Metadata



4. Return File List



5. Extract Code Elements
(Classes, Methods, etc.)



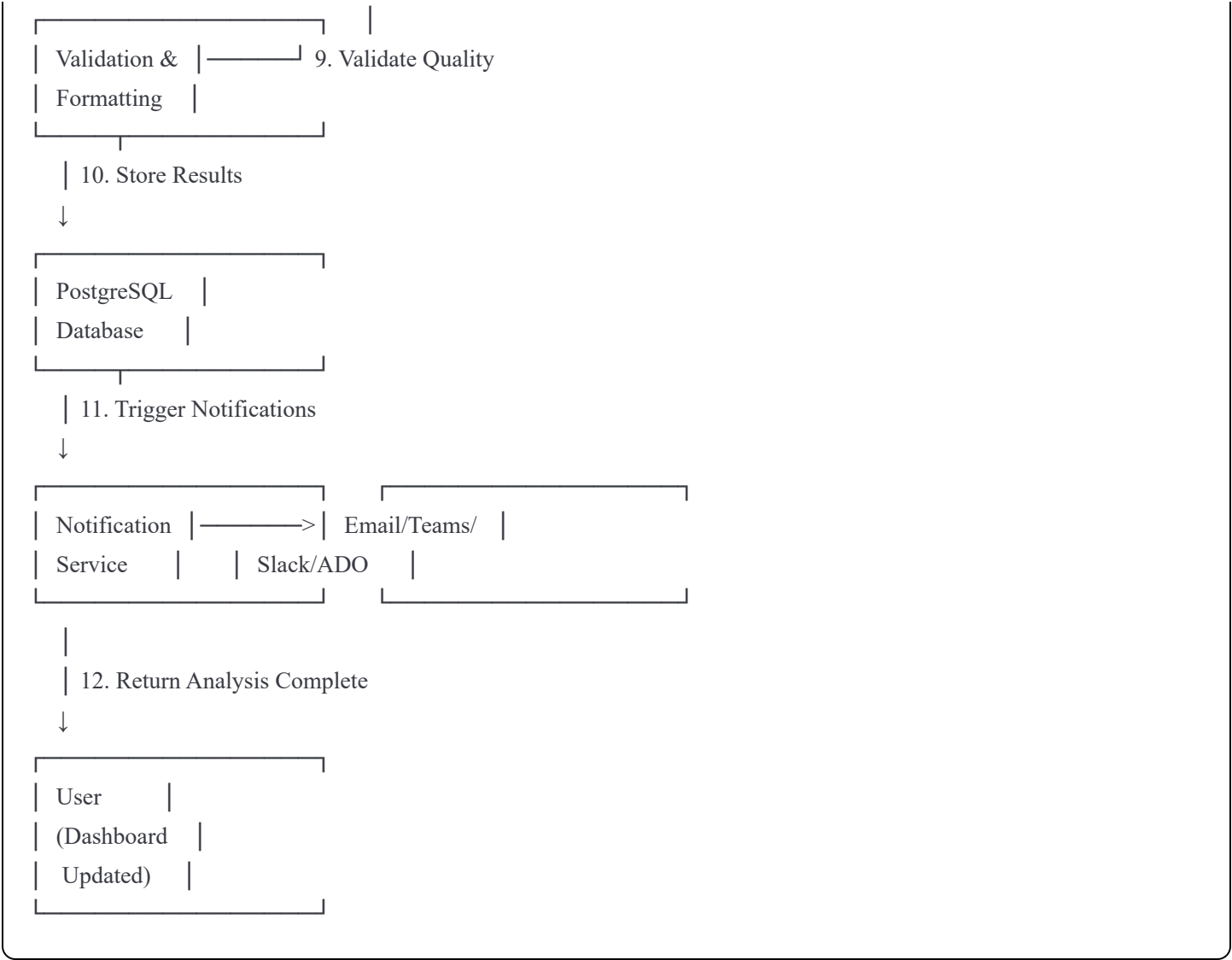
6. Identify Gaps



7. Generate Documentation

8. Return Generated Docs





API Design

REST API Endpoints

Authentication: All endpoints require Bearer token (JWT)

Base URL: <https://api.softura.com/doc-agent/v1>

Repository Management

POST	/api/repositories/scan
GET	/api/repositories
GET	/api/repositories/{id}
PUT	/api/repositories/{id}/settings
DELETE	/api/repositories/{id}
GET	/api/repositories/{id}/coverage
GET	/api/repositories/{id}/history

Analysis

POST /api/analysis/run
GET /api/analysis/{jobId}/status
GET /api/analysis/{jobId}/results
POST /api/analysis/{jobId}/cancel
GET /api/analysis/recent

Documentation

GET /api/documentation/gaps
GET /api/documentation/gaps/{id}
POST /api/documentation/generate
PUT /api/documentation/gaps/{id}/resolve
GET /api/documentation/suggestions
POST /api/documentation/suggestions/{id}/accept
POST /api/documentation/suggestions/{id}/reject

Reports

GET /api/reports/summary
GET /api/reports/repository/{id}
GET /api/reports/team/{teamId}
GET /api/reports/export?format=pdf|excel
POST /api/reports/schedule

Work Items (ADO Integration)

POST /api/workitems/create
GET /api/workitems/linked/{repositoryId}
PUT /api/workitems/{id}/update

Settings

GET /api/settings/templates
POST /api/settings/templates
PUT /api/settings/templates/{id}
DELETE /api/settings/templates/{id}
GET /api/settings/rules
PUT /api/settings/rules

Sample API Request/Response

POST /api/repositories/scan

Request:

```
json

{
  "source": "github",
  "repositoryUrl": "https://github.com/softura/core-api",
  "branch": "main",
  "scanType": "full",
  "options": {
    "generateDocs": true,
    "createWorkItems": true,
    "notifyOwners": true
  }
}
```

Response:

```
json

{
  "jobId": "550e8400-e29b-41d4-a716-446655440000",
  "status": "queued",
  "estimatedDuration": "5-10 minutes",
  "repositoryId": "12345",
  "createdAt": "2026-01-16T10:30:00Z",
  "links": {
    "status": "/api/analysis/550e8400-e29b-41d4-a716-446655440000/status",
    "results": "/api/analysis/550e8400-e29b-41d4-a716-446655440000/results"
  }
}
```

GET /api/analysis/{jobId}/results

Response:

```
json
```

```
{
  "jobId": "550e8400-e29b-41d4-a716-446655440000",
  "status": "completed",
  "repository": {
    "id": "12345",
    "name": "core-api",
    "url": "https://github.com/softura/core-api"
  },
  "summary": {
    "totalFiles": 850,
    "analyzedFiles": 650,
    "skippedFiles": 200,
    "overallCoverage": 67.5,
    "gapsFound": 245,
    "criticalGaps": 12,
    "highPriorityGaps": 45,
    "mediumPriorityGaps": 98,
    "lowPriorityGaps": 90
  },
  "coverageByType": {
    "classes": 72.3,
    "methods": 65.1,
    "properties": 80.5,
    "files": 55.0
  },
  "topGaps": [
    {
      "id": "gap-001",
      "file": "Services/PaymentService.cs",
      "line": 45,
      "element": "ProcessPayment",
      "type": "method",
      "severity": "critical",
      "message": "Public method missing XML documentation",
      "suggestion": "/// <summary>\n/// Processes a payment transaction...\n/// </summary>"
    }
  ],
  "completedAt": "2026-01-16T10:38:22Z",
  "duration": "8m 22s"
}
```

Database Schema

Tables

repositories

sql

```
CREATE TABLE repositories (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name VARCHAR(255) NOT NULL,  
  source VARCHAR(50) NOT NULL, -- 'github' or 'ado'  
  repository_url TEXT NOT NULL,  
  default_branch VARCHAR(100) DEFAULT 'main',  
  is_active BOOLEAN DEFAULT true,  
  last_scanned_at TIMESTAMP,  
  scan_frequency VARCHAR(50), -- 'manual', 'daily', 'weekly', 'on-commit'  
  settings JSONB,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  created_by VARCHAR(255),  
  
  CONSTRAINT unique_repo_url UNIQUE(repository_url)  
);  
  
CREATE INDEX idx_repositories_source ON repositories(source);  
CREATE INDEX idx_repositories_active ON repositories(is_active);  
CREATE INDEX idx_repositories_last_scanned ON repositories(last_scanned_at);
```

analysis_jobs

sql

```
CREATE TABLE analysis_jobs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  repository_id UUID NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,  
  status VARCHAR(50) NOT NULL, -- 'queued', 'running', 'completed', 'failed', 'cancelled'  
  scan_type VARCHAR(50), -- 'full', 'incremental', 'pr'  
  branch VARCHAR(100),  
  commit_sha VARCHAR(100),  
  started_at TIMESTAMP,  
  completed_at TIMESTAMP,  
  duration_seconds INTEGER,  
  error_message TEXT,  
  configuration JSONB,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  created_by VARCHAR(255),  
  
  CONSTRAINT chk_status CHECK (status IN ('queued', 'running', 'completed', 'failed', 'cancelled'))  
);  
  
CREATE INDEX idx_analysis_jobs_repository ON analysis_jobs(repository_id);  
CREATE INDEX idx_analysis_jobs_status ON analysis_jobs(status);  
CREATE INDEX idx_analysis_jobs_created ON analysis_jobs(created_at DESC);
```

analysis_results

sql

```
CREATE TABLE analysis_results (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  job_id UUID NOT NULL REFERENCES analysis_jobs(id) ON DELETE CASCADE,  
  repository_id UUID NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,  
  total_files INTEGER NOT NULL,  
  analyzed_files INTEGER NOT NULL,  
  skipped_files INTEGER NOT NULL,  
  overall_coverage DECIMAL(5,2),  
  total_gaps INTEGER NOT NULL,  
  critical_gaps INTEGER DEFAULT 0,  
  high_priority_gaps INTEGER DEFAULT 0,  
  medium_priority_gaps INTEGER DEFAULT 0,  
  low_priority_gaps INTEGER DEFAULT 0,  
  coverage_by_type JSONB, -- { "classes": 72.3, "methods": 65.1, ... }  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX idx_analysis_results_job ON analysis_results(job_id);  
CREATE INDEX idx_analysis_results_repository ON analysis_results(repository_id);  
CREATE INDEX idx_analysis_results_coverage ON analysis_results(overall_coverage);
```

documentation_gaps

sql

```

CREATE TABLE documentation_gaps (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id UUID NOT NULL REFERENCES analysis_jobs(id) ON DELETE CASCADE,
  repository_id UUID NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
  file_path TEXT NOT NULL,
  file_type VARCHAR(50), -- 'csharp', 'typescript', 'python', 'markdown'
  line_number INTEGER,
  element_name VARCHAR(500), -- class name, method name, etc.
  element_type VARCHAR(50), -- 'class', 'method', 'property', 'file', 'readme'
  gap_type VARCHAR(100), -- 'missing', 'incomplete', 'outdated', 'incorrect_format'
  severity VARCHAR(20) NOT NULL, -- 'critical', 'high', 'medium', 'low'
  message TEXT,
  current_documentation TEXT,
  ai_suggestion TEXT,
  suggestion_confidence DECIMAL(3,2), -- 0.00 to 1.00
  status VARCHAR(50) DEFAULT 'open', -- 'open', 'resolved', 'ignored', 'in_progress'
  resolved_at TIMESTAMP,
  resolved_by VARCHAR(255),
  work_item_id VARCHAR(100), -- ADO work item ID if created
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT chk_severity CHECK (severity IN ('critical', 'high', 'medium', 'low')),
  CONSTRAINT chk_status CHECK (status IN ('open', 'resolved', 'ignored', 'in_progress'))
);

CREATE INDEX idx_documentation_gaps_job ON documentation_gaps(job_id);
CREATE INDEX idx_documentation_gaps_repository ON documentation_gaps(repository_id);
CREATE INDEX idx_documentation_gaps_status ON documentation_gaps(status);
CREATE INDEX idx_documentation_gaps_severity ON documentation_gaps(severity);
CREATE INDEX idx_documentation_gaps_file ON documentation_gaps(file_path);
CREATE INDEX idx_documentation_gaps_element ON documentation_gaps(element_type, element_name);

```

documentation_templates

```
sql
```

```
CREATE TABLE documentation_templates (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name VARCHAR(255) NOT NULL,  
  description TEXT,  
  language VARCHAR(50), -- 'csharp', 'typescript', 'python', 'all'  
  element_type VARCHAR(50), -- 'class', 'method', 'property', 'file'  
  template_content TEXT NOT NULL,  
  variables JSONB, -- Variables that can be substituted in template  
  is_default BOOLEAN DEFAULT false,  
  is_active BOOLEAN DEFAULT true,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  created_by VARCHAR(255)  
);  
  
CREATE INDEX idx_templates_language ON documentation_templates(language);  
CREATE INDEX idx_templates_element_type ON documentation_templates(element_type);  
CREATE INDEX idx_templates_active ON documentation_templates(is_active);
```

coverage_history

sql

```

CREATE TABLE coverage_history (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  repository_id UUID NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
  job_id UUID NOT NULL REFERENCES analysis_jobs(id) ON DELETE CASCADE,
  measured_at TIMESTAMP NOT NULL,
  overall_coverage DECIMAL(5,2),
  class_coverage DECIMAL(5,2),
  method_coverage DECIMAL(5,2),
  property_coverage DECIMAL(5,2),
  file_coverage DECIMAL(5,2),
  total_gaps INTEGER,
  critical_gaps INTEGER,
  high_priority_gaps INTEGER,
  medium_priority_gaps INTEGER,
  low_priority_gaps INTEGER,

  CONSTRAINT unique_repo_measurement UNIQUE(repository_id, measured_at)
);

CREATE INDEX idx_coverage_history_repository ON coverage_history(repository_id);
CREATE INDEX idx_coverage_history_measured ON coverage_history(measured_at DESC);

```

users

```

sql

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) NOT NULL UNIQUE,
  full_name VARCHAR(255),
  ado_user_id VARCHAR(100),
  github_username VARCHAR(100),
  role VARCHAR(50) DEFAULT 'developer', -- 'admin', 'manager', 'developer'
  notification_preferences JSONB,
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role ON users(role);

```

notifications

sql

```
CREATE TABLE notifications (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  notification_type VARCHAR(50), -- 'email', 'teams', 'slack', 'in_app'  
  subject VARCHAR(500),  
  message TEXT,  
  severity VARCHAR(20), -- 'info', 'warning', 'critical'  
  related_entity_type VARCHAR(50), -- 'repository', 'gap', 'job'  
  related_entity_id UUID,  
  status VARCHAR(50) DEFAULT 'pending', -- 'pending', 'sent', 'failed', 'read'  
  sent_at TIMESTAMP,  
  read_at TIMESTAMP,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX idx_notifications_user ON notifications(user_id);  
CREATE INDEX idx_notifications_status ON notifications(status);  
CREATE INDEX idx_notifications_created ON notifications(created_at DESC);
```

ado_work_items

sql

```
CREATE TABLE ado_work_items (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  gap_id UUID REFERENCES documentation_gaps(id) ON DELETE CASCADE,  
  work_item_id INTEGER NOT NULL, -- ADO work item ID  
  work_item_url TEXT,  
  work_item_type VARCHAR(50), -- 'Task', 'Bug', 'User Story'  
  state VARCHAR(50), -- 'New', 'Active', 'Resolved', 'Closed'  
  assigned_to VARCHAR(255),  
  created_in_ado_at TIMESTAMP,  
  synced_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  CONSTRAINT unique_gap_work_item UNIQUE(gap_id)  
);  
  
CREATE INDEX idx_ado_work_items_gap ON ado_work_items(gap_id);  
CREATE INDEX idx_ado_work_items_state ON ado_work_items(state);
```

Sample Data

repositories

```
sql

INSERT INTO repositories (id, name, source, repository_url, default_branch, scan_frequency)
VALUES
('a1b2c3d4-e5f6-4a5b-8c9d-0e1f2a3b4c5d', 'core-api', 'github', 'https://github.com/softura/core-api', 'main', 'daily'),
('b2c3d4e5-f6a7-5b6c-9d0e-1f2a3b4c5d6e', 'web-portal', 'ado', 'https://dev.azure.com/softura/WebPortal/_git/web-portal', 'd
('c3d4e5f6-a7b8-6c7d-0e1f-2a3b4c5d6e7f', 'mobile-app', 'github', 'https://github.com/softura/mobile-app', 'main', 'weekly');
```

documentation_gaps

```
sql

INSERT INTO documentation_gaps (
    repository_id, file_path, file_type, line_number, element_name,
    element_type, gap_type, severity, message, ai_suggestion
)
VALUES (
    'a1b2c3d4-e5f6-4a5b-8c9d-0e1f2a3b4c5d',
    'Services/PaymentService.cs',
    'csharp',
    45,
    'ProcessPayment',
    'method',
    'missing',
    'critical',
    'Public method ProcessPayment is missing XML documentation',
    '/// <summary>
/// Processes a payment transaction for the specified order.
/// </summary>
/// <param name="orderId">The unique identifier of the order to process.</param>
/// <param name="amount">The payment amount in USD.</param>
/// <returns>A PaymentResult indicating success or failure of the transaction.</returns>
/// <exception cref="ArgumentException">Thrown when orderId is invalid.</exception>
/// <exception cref="PaymentException">Thrown when payment processing fails.</exception>'
);
```

Security Specifications

Authentication & Authorization

1. JWT-based Authentication

- OAuth 2.0 / OpenID Connect integration with Azure AD

- JWT tokens with 1-hour expiration
- Refresh token rotation
- Token revocation support

2. Role-Based Access Control (RBAC)

- **Admin:** Full system access, configuration management
- **Manager:** View all reports, manage team settings
- **Developer:** View assigned gaps, accept/reject suggestions
- **Viewer:** Read-only access to reports

3. API Security

- All endpoints require authentication
- Rate limiting: 100 requests/minute per user
- CORS configuration for allowed origins
- HTTPS only (TLS 1.2+)

Data Security

1. Secrets Management

- GitHub/ADO tokens stored in Azure Key Vault
- OpenAI API keys in Key Vault
- Database credentials in Key Vault
- No secrets in source code or configuration files

2. Data Encryption

- Data in transit: TLS 1.2+
- Data at rest: Azure Database encryption
- Encrypted backup storage

3. PII Handling

- No PII stored except user email (for notifications)
- GDPR compliance for EU users
- Data retention policies (2 years max)

Audit & Compliance

1. Audit Logging

- All API calls logged with user, timestamp, action

- Failed authentication attempts logged
- Data modification events logged
- Logs retained for 1 year

2. Compliance

- SOC 2 Type II considerations
- GDPR data subject rights (export, delete)
- Regular security scans and penetration testing

Performance Requirements

Response Time

- API endpoints: < 200ms (p95)
- Dashboard load: < 3 seconds
- Repository scan: < 5 minutes for 10K files
- AI documentation generation: < 10 seconds per file

Scalability

- Support 100 concurrent repository scans
- Handle 500 concurrent API requests
- Store data for 1000+ repositories
- Support 500+ active users

Availability

- 99.5% uptime SLA
- Scheduled maintenance windows (Saturday 2-4 AM EST)
- Automated failover for database
- Health check endpoints

Deployment Architecture

Environments

1. Development

- Local Docker Compose
- In-memory database option

- Mock external APIs
- Hot reload enabled

2. Staging

- Azure Container Apps
- Shared PostgreSQL instance
- Staging Azure OpenAI deployment
- Mirrors production configuration

3. Production

- Azure Container Apps (autoscaling)
- Azure Database for PostgreSQL (High Availability)
- Production Azure OpenAI deployment
- Application Insights monitoring
- Azure Front Door for global distribution

CI/CD Pipeline

yaml

Simplified pipeline structure

trigger:

branches:

include:

- main
- develop

stages:

- stage: Build

jobs:

- job: BuildBackend

steps:

- task: DotNetCoreCLI@2

inputs:

command: 'build'

projects: '**/*.csproj'

- task: DotNetCoreCLI@2

inputs:

command: 'test'

projects: '**/*Tests.csproj'

- job: BuildFrontend

steps:

- task: Npm@1

inputs:

command: 'install'

- task: Npm@1

inputs:

command: 'custom'

customCommand: 'run build'

- stage: Deploy_Staging

dependsOn: Build

condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/develop'))

jobs:

- deployment: DeployToStaging

environment: staging

strategy:

runOnce:

deploy:

steps:

- task: Docker@2

```
    inputs:
      command: 'buildAndPush'
- task: AzureContainerApps@1
  inputs:
    command: 'deploy'

- stage: Deploy_Production
  dependsOn: Build
  condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
  jobs:
    - deployment: DeployToProduction
      environment: production
      strategy:
        runOnce:
          deploy:
            steps:
              - task: Docker@2
                inputs:
                  command: 'buildAndPush'
              - task: AzureContainerApps@1
                inputs:
                  command: 'deploy'
```

Success Criteria

Business Metrics

1. Documentation Coverage Improvement

- Target: Increase average coverage from current baseline to 80% within 6 months
- Measurement: Monthly coverage snapshots across all repositories

2. Rework Reduction

- Target: Reduce documentation-related rework by 25% within 3 months
- Measurement: ADO work items tagged as "documentation rework"

3. Developer Adoption

- Target: 80% of developers actively use the tool within 2 months
- Measurement: Active users per week, API usage metrics

4. Time Savings

- Target: Reduce time spent writing documentation by 40%

- Measurement: Developer surveys, time tracking

5. Knowledge Transfer

- Target: Reduce onboarding time for new developers by 30%
- Measurement: Time to first productive commit for new hires

Technical Metrics

1. System Performance

- API response time p95 < 200ms
- Repository scan completion < 5 minutes for 10K files
- Dashboard load time < 3 seconds
- AI generation accuracy > 90% for simple methods

2. Reliability

- System uptime > 99.5%
- Failed job rate < 1%
- Zero critical security vulnerabilities

3. Scalability

- Support 100 concurrent scans without degradation
- Handle 500+ concurrent users
- Process 1M+ files across all repositories

User Satisfaction

1. Developer Satisfaction

- Target: > 4.0/5.0 average rating
- Measurement: In-app feedback surveys

2. Generated Documentation Quality

- Target: > 80% acceptance rate for AI suggestions
- Measurement: Accept/reject ratio

3. Support Tickets

- Target: < 5 support tickets per month
 - Measurement: ADO support board
-

Acceptance Criteria by Feature

AC-1: Repository Scanning (FR-1)

Given a user has configured a GitHub or ADO repository

When they initiate a scan

Then:

- ✓ System authenticates successfully with the repository
- ✓ System retrieves the complete file tree
- ✓ System filters files by supported types (.cs, .js, .ts, .py, .md)
- ✓ System completes scan in < 5 minutes for 10K files
- ✓ System handles API rate limits gracefully with exponential backoff
- ✓ System logs all operations to Application Insights

And the scan encounters an error

Then:

- ✓ System logs detailed error information
- ✓ System notifies the user via in-app notification
- ✓ System marks the job as "failed" with error message
- ✓ System allows retry without re-scanning completed files

AC-2: Code Documentation Analysis (FR-2)

Given a C# file with classes and methods

When the analysis engine processes the file

Then:

- ✓ System identifies all public classes without XML documentation
- ✓ System identifies all public methods without XML documentation
- ✓ System identifies methods with incomplete parameter documentation
- ✓ System identifies missing return value documentation
- ✓ System calculates coverage percentage (documented/total * 100)
- ✓ System assigns severity based on visibility (public = critical)

Given a TypeScript file with functions

When the analysis engine processes the file

Then:

- ✓ System identifies functions without JSDoc comments
- ✓ System validates parameter documentation completeness
- ✓ System validates return type documentation
- ✓ System flags missing examples in public APIs

Given a file where code was modified but documentation wasn't

When the analysis compares to previous scan

Then:

- ✓ System detects stale documentation
- ✓ System flags as "outdated" with high severity
- ✓ System suggests review of documentation

AC-3: README Analysis (FR-3)

Given a repository without README.md

When the analysis runs

Then:

- ✓ System flags missing README with critical severity
- ✓ System generates a README template based on repository type
- ✓ System includes standard sections (Overview, Installation, Usage)

Given a README.md with missing sections

When the analysis runs

Then:

- ✓ System identifies missing sections (Installation, Usage, etc.)
- ✓ System calculates README quality score (0-100)
- ✓ System suggests specific sections to add

Given a README.md with external links

When the analysis runs

Then:

- ✓ System validates HTTP links return 200 OK
- ✓ System flags broken links with medium severity
- ✓ System suggests link fixes or removal

AC-4: AI Documentation Generation (FR-4)

Given an undocumented C# method

When AI generation is triggered

Then:

- ✓ System generates valid XML documentation comment
- ✓ Generated docs include <summary> tag
- ✓ Generated docs include all <param> tags
- ✓ Generated docs include <returns> tag if method returns value
- ✓ Generated docs include <exception> tags for thrown exceptions
- ✓ Generated docs are technically accurate (90%+ accuracy)
- ✓ Generated docs use appropriate technical terminology

Given a TypeScript function with complex logic

When AI generation is triggered

Then:

- ✓ System generates JSDoc comment
- ✓ Generated docs include @param for all parameters
- ✓ Generated docs include @returns tag
- ✓ Generated docs include @example if function is public API
- ✓ Generated docs include @throws for error conditions

Given a Python class

When AI generation is triggered

Then:

- ✓ System generates proper docstring format
- ✓ Generated docs follow Google or NumPy style
- ✓ Generated docs include Args, Returns, Raises sections
- ✓ Generated docs are properly indented

Given AI generates documentation for a developer

When developer reviews the suggestion

Then:

- ✓ Developer can view side-by-side (current vs. suggested)
- ✓ Developer can accept suggestion with one click

- ✓ Developer can reject suggestion with one click
- ✓ Developer can edit suggestion before accepting
- ✓ System tracks accept/reject ratios for quality metrics

AC-5: Dashboard (FR-5)

Given a user opens the dashboard

When the page loads

Then:

- ✓ Dashboard loads in < 3 seconds
- ✓ Overall coverage percentage is displayed prominently
- ✓ Trend chart shows coverage over last 30 days
- ✓ Top 10 repositories with lowest coverage are shown
- ✓ Critical gaps count is highlighted

Given a user selects a specific repository

When viewing repository details

Then:

- ✓ Coverage breakdown by file type is shown
- ✓ Historical trend for this repository is displayed
- ✓ List of gaps is displayed with filtering options
- ✓ User can filter by severity, file type, status
- ✓ User can sort by severity, file name, date

Given a manager views team performance

When accessing team view

Then:

- ✓ Coverage metrics by team member are shown
- ✓ Contributions to documentation improvements are tracked
- ✓ Leaderboard shows top contributors
- ✓ Export to Excel/PDF is available

Given a user on mobile device

When accessing dashboard

Then:

- ✓ Layout adapts to mobile screen size

- ✓ All critical information is accessible
- ✓ Charts are readable on small screens

AC-6: Azure DevOps Integration (FR-6)

Given a critical documentation gap is found

When auto-create work items is enabled

Then:

- ✓ System creates ADO work item within 2 minutes
- ✓ Work item title clearly describes the gap
- ✓ Work item description includes file path and line number
- ✓ Work item includes AI-generated suggestion
- ✓ Work item is assigned to file's last committer
- ✓ Work item priority matches gap severity
- ✓ Work item links to source code file

Given a pull request is created

When PR analysis is triggered

Then:

- ✓ System analyzes documentation changes within 2 minutes
- ✓ System posts comment on PR with coverage delta
- ✓ System flags if critical documentation is missing
- ✓ System blocks PR merge if configured (quality gate)
- ✓ Comment includes actionable feedback

Given a CI/CD pipeline runs

When documentation task executes

Then:

- ✓ Task completes in < 10 minutes
- ✓ Task generates coverage report
- ✓ Report published to pipeline artifacts
- ✓ Build fails if coverage drops below threshold (if configured)
- ✓ Pipeline summary shows coverage percentage

AC-7: Notifications (FR-7)

Given a critical gap is discovered

When the analysis completes

Then:

- ✓ Email sent to assigned developer within 5 minutes
- ✓ Email includes gap description and file link
- ✓ Email includes AI-generated suggestion
- ✓ Email has clear call-to-action

Given a user has configured Teams notifications

When their team's coverage drops below threshold

Then:

- ✓ Teams message posted to team channel
- ✓ Message includes coverage trend
- ✓ Message includes top gaps to address

Given a user prefers digest mode

When weekly summary time arrives

Then:

- ✓ User receives single email with all gaps
- ✓ Email groups gaps by severity
- ✓ Email includes links to dashboard
- ✓ Email shows team progress

Given a user wants to unsubscribe

When clicking unsubscribe link

Then:

- ✓ User is unsubscribed immediately
- ✓ Confirmation message is shown
- ✓ User can re-subscribe from settings

AC-8: Templates and Standards (FR-8)

Given an admin creates a custom template

When saving the template

Then:

- ✓ Template is validated for syntax errors
- ✓ Template variables are identified

- ✓ Template is saved to database
- ✓ Template appears in template library

Given a C# repository uses Softura coding standards

When generating documentation

Then:

- ✓ System applies Softura template
- ✓ Generated docs follow organizational style
- ✓ Generated docs include required sections

Given a developer violates style guidelines

When analysis detects violation

Then:

- ✓ Violation is flagged as medium severity
- ✓ Suggestion includes corrected format
- ✓ Link to style guide is provided

AC-9: Historical Tracking (FR-9)

Given repository has been scanned multiple times

When viewing coverage history

Then:

- ✓ Line chart shows coverage over time
- ✓ Data points include last 90 days minimum
- ✓ Significant events are annotated (major releases, etc.)
- ✓ User can export historical data as CSV

Given a team wants to compare performance

When viewing team comparison report

Then:

- ✓ Side-by-side comparison of multiple teams
- ✓ Metrics include coverage, gaps resolved, response time
- ✓ Trend indicators show improvement/decline

Given historical data exceeds 2 years

When retention policy runs

Then:

- ✓ Data older than 2 years is archived
- ✓ Summarized metrics are retained
- ✓ Raw gap data is deleted per policy

AC-10: Search and Discovery (FR-10)

Given a user searches for "payment processing"

When search executes

Then:

- ✓ Results returned in < 2 seconds
- ✓ Results include relevant documentation
- ✓ Results rank by relevance
- ✓ Search terms highlighted in results
- ✓ Results show snippet with context

Given a user uses semantic search

When searching conceptually similar terms

Then:

- ✓ Results include semantically related content
 - ✓ Exact keyword match not required
 - ✓ Results grouped by relevance score
-

Non-Functional Requirements

Usability

- Dashboard intuitive for non-technical users
- No more than 3 clicks to reach any feature
- Consistent UI/UX across all pages
- Accessible (WCAG 2.1 Level AA compliance)
- Keyboard navigation support

Maintainability

- Code coverage > 80% for backend
- Code coverage > 70% for frontend

- Comprehensive API documentation (Swagger)
- Inline code comments for complex logic
- Automated dependency updates (Dependabot)

Observability

- Application Insights integration
- Custom dashboards for key metrics
- Alert rules for critical failures
- Distributed tracing for API calls
- Log aggregation and search

Disaster Recovery

- Daily automated backups
 - Point-in-time recovery capability
 - Backup restoration tested quarterly
 - RPO (Recovery Point Objective): 24 hours
 - RTO (Recovery Time Objective): 4 hours
-

Project Timeline

Phase 1: Foundation (Weeks 1-3)

- Week 1: Project setup, architecture design, database schema
- Week 2: GitHub/ADO API integration, authentication/authorization
- Week 3: Basic code parsing (C#), database models, repository pattern

Deliverables: Working API that can scan a repository and store basic metadata

Phase 2: Core Analysis (Weeks 4-6)

- Week 4: Documentation analysis engine (C#), gap detection logic
- Week 5: Azure OpenAI integration, prompt engineering
- Week 6: AI documentation generation, suggestion engine

Deliverables: API can analyze code, detect gaps, generate suggestions

Phase 3: Frontend & UX (Weeks 7-9)

- Week 7: React app setup, dashboard layout, basic components
- Week 8: Repository management UI, gap browsing, filtering
- Week 9: Charts and visualizations, responsive design

Deliverables: Functional web application with all major views

Phase 4: ADO Integration (Week 10)

- Work item creation API
- PR comment integration
- Pipeline task development
- Testing and refinement

Deliverables: Seamless ADO integration

Phase 5: Polish & Deploy (Weeks 11-12)

- Week 11: Bug fixes, performance optimization, security hardening
- Week 12: Documentation, deployment automation, user training

Deliverables: Production-ready application

Post-Launch: Iteration

- Gather user feedback
- Implement P1/P2 features
- Expand language support
- Enhance AI quality

Testing Strategy

Unit Testing

- **Backend:** xUnit, Moq for mocking
- **Frontend:** Jest, React Testing Library
- **Target:** > 80% code coverage

Integration Testing

- Test GitHub/ADO API interactions
- Test database operations
- Test AI service integration
- Use test doubles for external services

End-to-End Testing

- Playwright or Selenium for UI testing
- Test critical user journeys
- Automated E2E tests in CI/CD pipeline

Performance Testing

- Load testing with k6 or JMeter
- Test with 100 concurrent scans
- Measure API response times under load
- Database query optimization

Security Testing

- OWASP ZAP for vulnerability scanning
 - Dependency scanning (Snyk, Dependabot)
 - Secret scanning in repositories
 - Penetration testing before production
-

Risks and Mitigations

Risk 1: AI Quality Issues

Risk: Generated documentation is inaccurate or unhelpful

Impact: High - Users lose trust in the system

Mitigation:

- Implement confidence scoring
- Flag low-confidence suggestions for human review
- Continuous prompt engineering and improvement

- A/B testing of prompts
- User feedback loop to improve quality

Risk 2: API Rate Limiting

Risk: GitHub/ADO APIs throttle requests

Impact: Medium - Scans take longer or fail

Mitigation:

- Implement exponential backoff
- Respect rate limit headers
- Cache API responses where possible
- Batch requests efficiently
- Consider GitHub Apps for higher limits

Risk 3: OpenAI Cost Overruns

Risk: AI API costs exceed budget

Impact: High - Project becomes unsustainable

Mitigation:

- Set monthly spending limits
- Implement token counting and optimization
- Cache common patterns
- Use cheaper models for simple tasks
- Monitor costs daily with alerts

Risk 4: Adoption Resistance

Risk: Developers don't use the tool

Impact: High - No business value realized

Mitigation:

- Early user involvement in design
- Gamification and incentives
- Executive sponsorship
- Make tool easy and valuable
- Gradual rollout with champions

Risk 5: Performance Degradation

Risk: System slows down as repositories grow

Impact: Medium - User frustration

Mitigation:

- Design for scalability from day one
 - Implement caching aggressively
 - Use database indexing properly
 - Horizontal scaling capability
 - Regular performance testing
-

Success Metrics Dashboard

Track these KPIs in a dashboard visible to stakeholders:

Adoption Metrics

- Active users (daily/weekly/monthly)
- Repositories connected
- Scans performed per day
- AI suggestions generated
- Suggestions accepted vs. rejected

Quality Metrics

- Average documentation coverage across all repositories
- Coverage trend (week-over-week)
- Critical gaps remaining
- Mean time to resolution (gaps)

Business Impact

- Documentation-related rework incidents (ADO tags)
- Developer time spent on documentation (surveys)
- New hire onboarding time
- Support tickets related to missing documentation

System Health

- API uptime percentage
 - Average scan duration
 - API response times (p50, p95, p99)
 - Error rates
 - OpenAI API costs
-

Appendix A: Glossary

- **Coverage:** Percentage of code elements that have documentation
 - **Gap:** A missing, incomplete, or outdated documentation element
 - **Element:** A code construct (class, method, property, file, etc.)
 - **Severity:** Importance level of a gap (critical, high, medium, low)
 - **Suggestion:** AI-generated documentation draft
 - **Template:** Predefined documentation structure
 - **POD:** Product-Oriented Development (Softura framework)
 - **ADO:** Azure DevOps
 - **PR:** Pull Request
 - **CI/CD:** Continuous Integration / Continuous Deployment
-

Appendix B: References

- Azure OpenAI Documentation: <https://learn.microsoft.com/en-us/azure/ai-services/openai/>
 - GitHub API Documentation: <https://docs.github.com/en/rest>
 - Azure DevOps REST API: <https://learn.microsoft.com/en-us/rest/api/azure/devops>
 - Roslyn Documentation: <https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/>
 - Material-UI: <https://mui.com/>
 - PostgreSQL Documentation: <https://www.postgresql.org/docs/>
-

Appendix C: Contact Information

Project Sponsor: Ketan (COO, Softura)
Developer: [Your Nephew's Name]
Technical Advisor: [If applicable]
Timeline: 12 weeks
Budget: [To be determined based on Azure costs]

Document Version Control

Version	Date	Author	Changes
1.0	2026-01-16	Claude	Initial requirements document created

END OF REQUIREMENTS DOCUMENT